TransferFuzz-Pro: Large Language Model Driven Code Debugging Technology for Verifying Propagated Vulnerability

Siyuan Li, Kaiyu Xie, Yuekang Li, Hong Li, Yimo Ren, Limin Sun, and Hongsong Zhu, *Member, IEEE*

***Abstract*—Code reuse in software development frequently facilitates the spread of vulnerabilities, leading to imprecise scopes of affected software in CVE reports. Traditional methods focus primarily on detecting reused vulnerability code in target software but lack the ability to confirm whether these vulnerabilities can be triggered in new software contexts. In previous work, we introduced the TransferFuzz framework to address this gap by using historical trace-based fuzzing. However, its effectiveness is constrained by the need for manual intervention and reliance on source code instrumentation. To overcome these limitations, we propose TransferFuzz-Pro, a novel framework that integrates Large Language Model (LLM)-driven code debugging technology. By leveraging LLM for automated, human-like debugging and Proof-of-Concept (PoC) generation, combined with binary-level instrumentation, TransferFuzz-Pro extends verification capabilities to a wider range of targets. Our evaluation shows that TransferFuzz-Pro is significantly faster and can automatically validate vulnerabilities that were previously unverifiable using conventional methods. Notably, it expands the number of affected software instances for 15 CVE-listed vulnerabilities from 15 to 53 and successfully generates PoCs for various Linux distributions. These results demonstrate that TransferFuzz-Pro effectively verifies vulnerabilities introduced by code reuse in target software and automatically generation PoCs.**

***Index Terms*—Vulnerability verification, fuzzing, large language model, code reuse.**

Received 29 November 2024; revised 15 May 2025; accepted 22 June 2025. Date of publication 3 July 2025; date of current version 15 August 2025. This work was supported in part by the National Key Research and Development Program and Development Program of China under Grant 2022YFB3103904 and in part by the Youth Innovation Promotion Association CAS. The associate editor coordinating the review of this article and approving it for publication was T. Zhang. *(Siyuan Li and Kaiyu Xie contributed equally to this work.) (Corresponding authors: Yimo Ren; Yuekang Li.)*

Siyuan Li and Kaiyu Xie are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100089, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 1000893, China (e-mail: lisiyuan@iie.ac.cn; xiekaiyu@iie.ac.cn).

Yuekang Li is with the School of Computer Science and Engineering, The University of New South Wales, Sydney, NSW 20524, Australia (e-mail: yuekang.li@unsw.edu.au).

Hong Li, Yimo Ren, Limin Sun, and Hongsong Zhu are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100089, China (e-mail: lihong@iie.ac.cn; renyimo@iie.ac.cn; sunlimin@iie.ac.cn; zhuhongsong@iie.ac.cn).

Digital Object Identifier 10.1109/TSE.2025.3584774

I. Introduction

CODE reuse is integral to modern software development, enhancing efficiency and functionality. Open-source repositories and third-party libraries, hosted on platforms like GitHub [1], SourceForge [2], and Vcpkg [3], enable developers to integrate readily available code into new projects, allowing them to focus on innovation rather than reinventing foundational components. A 2024 Synopsys report [4] found that 96% of audited software incorporates code from at least one third-party library, highlighting the prevalence of code reuse in the industry.

However, code reuse also introduces risks. Shared code can propagate vulnerabilities across multiple software projects [5]. While reused code often benefits from updates, flaws may persist and spread as the code is integrated into new systems. Although researchers disclose vulnerabilities to platforms like CVE [6] and NVD [7], the full scope of affected software is often inadequately assessed. This study investigates the propagation of vulnerabilities in C/C++ binaries, where the impact may extend far beyond initial vulnerability reports [5].

Recent research has increasingly focused on detecting and verifying vulnerabilities caused by code reuse. Existing methods can be broadly categorized into two types: code similarity detection and fuzzing-based approaches. Both aim to identify the propagation of vulnerabilities through code reuse. Code similarity detection methods [5], [8], [9], [10] build codebases and identify vulnerable functions in target software via similarity matching. Some methods [11], [12], [13], [14] further assess whether these reused vulnerable functions have been patched. However, they primarily detect the presence of vulnerable code without confirming its triggerability in the target software, often resulting in false positives (Problem 1, **P1**). Fuzzing-based methods [15], [16], [17], [18] aim to generate proof-of-concepts (PoCs) for patch testing and crash reproduction. Despite their promise, these methods face limitations, such as requiring hours to verify a single vulnerability and struggling to trigger vulnerabilities that involve complex logic [16] (Problem 2, **P2**).

In our previous work TransferFuzz [19], we used historical traces to deal with the two problems. In detail, we collect some runtime information of the **basic binary** (the vulnerable binary detailed in CVE reports) and extract the historical traces to

guide the fuzzing process for the **target binary** (new binary that reused the vulnerable function of the basic binary). By using historical traces, we can achieve faster triggering of vulnerabilities in new software and trigger complex logic vulnerabilities (**for P2**). By generating PoCs for new software, it is possible to automatically verify the vulnerability is an actual threat to the new software (**for P1**). The evaluation results demonstrate that TransferFuzz can validate vulnerabilities that were previously unverifiable through existing techniques like DAFL [20] or SelectFuzz [21].

Despite its potential, TransferFuzz [19] faces two key challenges limiting its utility. First, like existing directed fuzzing approaches, it requires manual creation of fuzz drivers, necessitating manual analysis to determine program option combinations that reach the target function, reducing its usability (Challenge 1, **C1**). Second, TransferFuzz relies on source code instrumentation, requiring complete compilation of the target software before fuzzing, which restricts the range of software it can verify (Challenge 2, **C2**).

Our evaluation demonstrates that TransferFuzz-Pro effectively automates vulnerability validation. It verified more vulnerabilities than existing methods and achieved a speedup of 2.5x to 26.2x. Additionally, TransferFuzz needs to manually analyze the options required for fuzzing for all test items. In contrast, TransferFuzz-Pro can automatically generate 85% of the options within 90 seconds (**for C1**). Notably, it expanded the impacted software scope for 15 vulnerabilities listed in CVE reports, increasing the number of affected binaries from 15 to 53, highlighting its effectiveness in identifying propagated vulnerable code. Furthermore, TrasnferFuzz-Pro successfully generated PoCs for various Linux distributions, which TrasnferFuzz could not verify due to the absence of source code (**for C2**).

Our evaluation demonstrates that TransferFuzz-Pro effectively automates vulnerability validation. It verified more vulnerabilities than existing methods and achieved a speedup of 2.5x to 26.2x. Additionally, TransferFuzz needs to manually analyze the options required for fuzzing for all test items. In contrast, TransferFuzz-Pro can automatically generate 85% of the options within 90 seconds (**for C1**). Notably, it expanded the impacted software scope for 15 vulnerabilities listed in CVE reports, increasing the number of affected binaries from 15 to 53, highlighting its effectiveness in identifying propagated vulnerable code. Furthermore, TrasnferFuzz-Pro successfully generated proof-of-concepts (PoCs) for various affected Linux distributions, which TrasnferFuzz was unable to verify due to the lack of source code (**for C2**).

The contributions of this paper are summarized as follows:

- We present TransferFuzz-Pro, the first fully automated fuzzing-based framework for verifying propagated vulnerable code.
- We introduce an LLM-driven code debugging technique and are the first to automate program option combination generation for directed fuzzing tasks.
- We analyze the software impact of 15 CVE-listed vulnerabilities, identifying 38 additional affected binaries that were previously unreported.

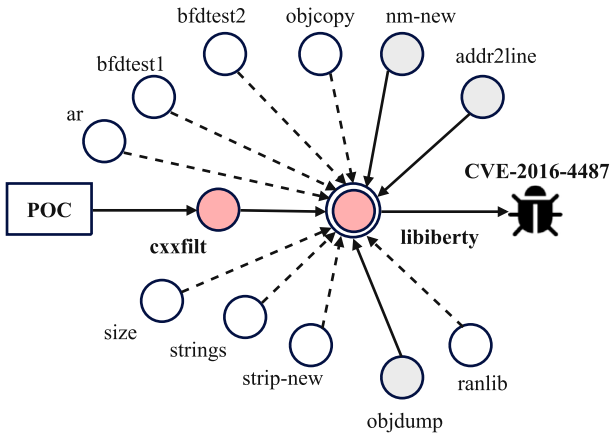


Fig. 1. Propagated vulnerability code. Red nodes are vulnerability information in CVE reports and historical research, white nodes are binaries that reuse vulnerable code, and gray nodes are binaries affected by the vulnerability.

- We generate Proof-of-Concepts (PoCs) for multiple Linux distributions impacted by these vulnerabilities, demonstrating the effectiveness of TransferFuzz-Pro.

II. Motivation

We illustrate our motivation and insights through several examples. First, we demonstrate how vulnerability code propagates. Then, we outline the objectives of program option combination generation and historical trace-guided fuzzing.

A. Vulnerability Code Propagation

Fig. 1 illustrates the CVE-2016-4487 vulnerability in *libiberty* as documented in the CVE database [6] and NVD reports [7]. AFLGo [15] applies fuzzing to the *cxxfilt* binary and generates a PoC that triggers this vulnerability. Previous studies [18], [20] assumed that CVE-2016-4487 was limited to *cxxfilt*, overlooking its potential impact on other binaries. Through manual analysis, we identified 11 additional binaries reusing the vulnerable *libiberty* code. However, due to variations in input file formats among these binaries, a universal PoC cannot be constructed, complicating the process of determining which binaries are really affected. By manually crafting specific inputs, we successfully triggered the vulnerability in *objdump*, *nm-new*, and *addr2line*. This case highlights how vulnerabilities in C/C++ binaries can propagate via code reuse, revealing that the reach of a vulnerability often surpasses the scope outlined in vulnerability reports or prior research.

However, not all instances of reused vulnerable code lead to the propagation of vulnerabilities. As shown in Fig. 1, eight binaries were unaffected by CVE-2016-4487. Although these binaries included the vulnerable code, the associated functions were not reachable. Beyond reachability, other factors also influence vulnerability propagation. For example, CVE-2016-10095 describes a stack-based buffer overflow in *LibTIFF*, where a buffer overflow occurs if the second parameter, *tag*, of the function *_TIFFVGetField* is set to 0x13d. While *OpenJPEG* integrates the vulnerable *LibTIFF* code, it remains unaffected

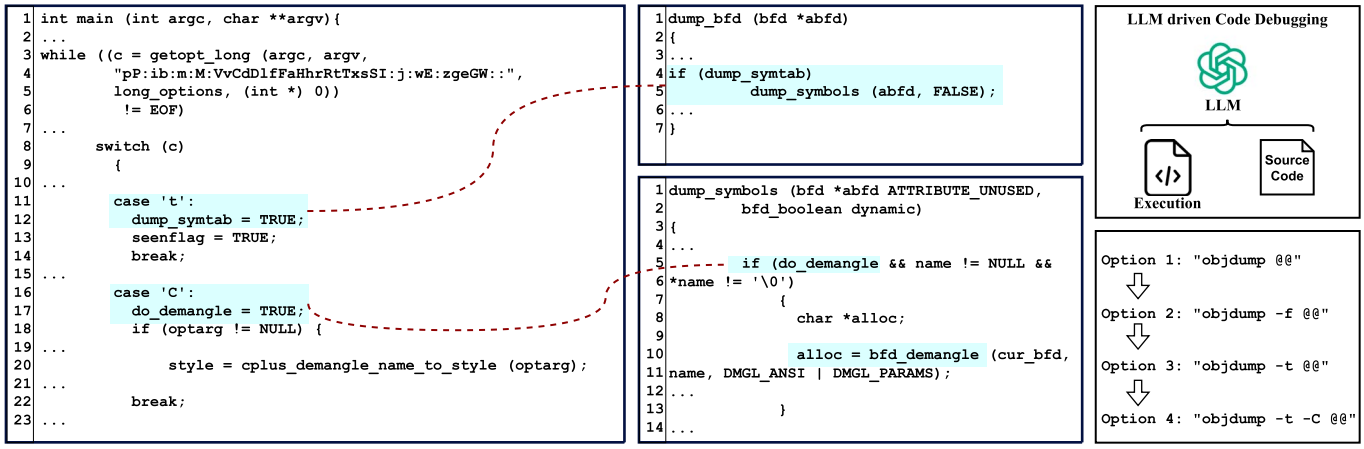


Fig. 2. A motivation example. The blue code is the option processing code and the corresponding key constraint which is impacted by option combinations. We used LLM-driven code debugging and generated the required option combination for objdump after four iterations.

because the variable *tag* is hard-coded, making the critical variables required to trigger the vulnerability uncontrollable. This highlights a limitation of both Code Reuse Detection and Path Reachability Detection methods, as neither can reliably determine whether vulnerable functions are reachable or whether critical variables are controllable.

B. Program Option Combination Generation

To generate a PoC for a target binary and verify the propagation of vulnerable code, it is essential to identify the program option combinations that enable the program to reach the vulnerable function. For instance, in our prior research on TransferFuzz, manual analysis revealed that the *objdump* binary requires the "-t -C" command-line parameters to execute the vulnerable function *cplus_demangle* and its sub-functions. Given that *objdump* supports numerous command-line parameters and combinations, this process can be complex. Previous works [22], [23] attempted to extract relationships between options from help manuals to generate option combinations for broader vulnerability discovery. However, no existing research addresses option combination generation specifically for directed fuzzing. Current methods [18], [20], [21] still rely on manual analysis to determine the option combinations necessary to reach the target function.

Large Language Models (LLMs) have recently been widely adopted in software engineering. Thanks to this code debugging technology, we have realized the first directional fuzz testing tool with automatic options generation. In this work, we leverage LLMs to simulate the human debugging process and are the first to automated generate option combinations for directed fuzzing tasks. Based on program execution results, the LLM analyzes the command-line parameters required to reach the next target point and iteratively continues debugging until the vulnerable code is reached. The command-line parameters identified during the debugging process are then combined to form the final program option combination.

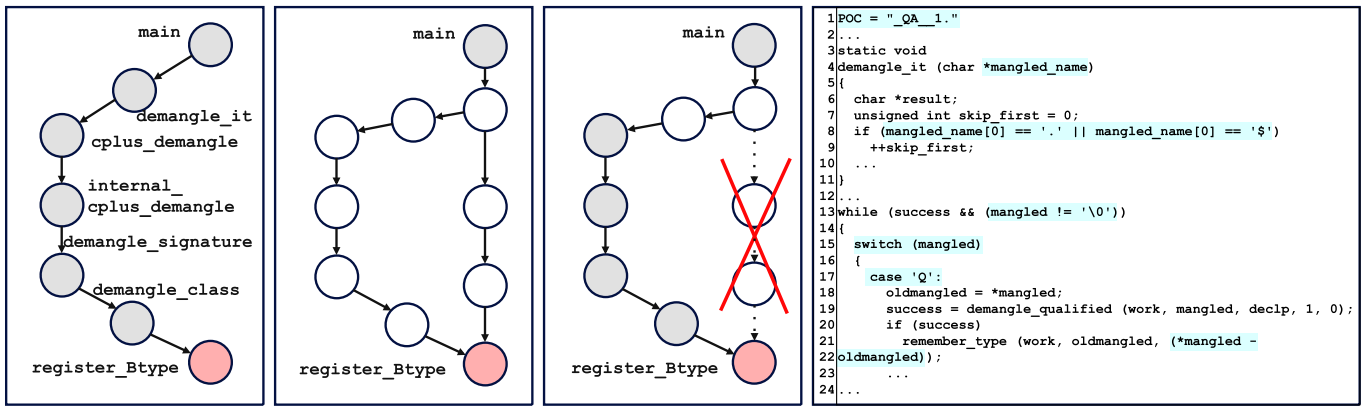
As illustrated in Fig. 2, the main function often includes option-processing code that sets flag variables (e.g., *dump_symtab* and *do_demangle*) or calls specific functions (e.g., *cplus_demangle_name_to_style*) based on different option combinations, influencing subsequent execution paths. By utilizing the LLM's inherent knowledge and code semantics understanding, we allow it to analyze code semantics and incorporate feedback from debugger outputs. Through multiple iterations, the LLM autonomously generates the program option combinations needed for *objdump* to reach the vulnerable code.

Interestingly, in addition to the "-t -C" combination previously identified through manual analysis, the LLM discovered that both "-t -C" and "-t -e" can reach the vulnerable code and successfully trigger the vulnerability. This demonstrates that LLM-driven generation of program option combinations can, in some cases, surpass the comprehensiveness of human analysis.

C. Historical Trace Guided Fuzzing

By combining program options, fuzzing can be automatically invoked for vulnerability verification. FuzzGuard [24] utilizes historical fuzzing data within the same software to guide new fuzzing efforts, while VulScope [25] leverages historical fuzzing data from different software versions for the same purpose. Building on these approaches, we propose a method that uses historical fuzzing data from different software sharing reused code regions to guide fuzzing. Our approach involves executing PoCs on the basic binary to extract historical traces, which are then used to guide fuzzing on a new target binary to trigger vulnerabilities. Specifically, we focus on two types of historical traces: function-level traces and key-byte traces.

Function-level Traces: Fig. 3(a) illustrates how executing *cxxfilt* with the PoC for CVE-2016-4487 reveals the sequence of function calls that trigger the vulnerability, referred to as function-level traces. These traces enable a shift from aimlessly fuzzing, as shown in Fig. 3(b) for *objdump*, to a more focused approach depicted in Fig. 3(c). This targeted method prioritizes paths relevant to the vulnerability, significantly accelerating the fuzzing process. Through static analysis, we identified 3,150 unique function call paths in *objdump* from the main function to the vulnerable function. However, leveraging function-level



(a) POC trace of cxxfilt (function level)

(b) Default trace of objdump (function level)

(c) Transferred trace of objdump (function level)

(d) Source Code of cxxfilt (key-bytes trace)

Fig. 3. A motivation example. The red node is the vulnerability function of CVE-2016-4487. Gray nodes are functions known to be on the vulnerability-triggering path, and white nodes are functions that may pass through. The blue code in 1(d) is the code in *cxxfilt* that accesses PoC related bytes.

traces from *cxxfilt*, we narrowed this down to 42 relevant paths, discarding the rest.

Key-bytes Traces: Fig. 3(d) highlights how specific key bytes in the *cxxfilt* PoC can bypass branch condition constraints in reused code (as shown in the highlighted code). These branches, present in the reused code of the target binary, can impede fuzzing efficiency. By extracting these bytes as key-byte traces, the fuzzing process is significantly accelerated, improving the effectiveness of vulnerability verification.

III. Methodology

A. Overview

The workflow of TransferFuzz-Pro, illustrated in Fig. 4, comprises three modules: 1) the option combination generation module, 2) the historical trace extraction module, and 3) the trace-guided fuzzing module. We assume that code similarity detection methods, such as LibAM [5], have identified that the target binary reuses vulnerable code from a basic binary. TransferFuzz-Pro aims to further verify whether the vulnerability can be triggered in the target binary. Given a basic binary with its PoC and the target binary as inputs, TransferFuzz-Pro determines if the target binary is affected by the vulnerability. If so, generate a PoC for the target binary.

The historical trace extraction module collects function-level traces and key-byte traces to guide fuzzing. First, the corresponding PoC is executed on the basic binary to capture runtime function call sequences, which serve as function-level traces for guiding fuzzing in the target binary. Additionally, taint analysis is used to identify bytes in the PoC that directly influence conditional statements in the reused code. These key bytes are compiled into a dictionary to guide mutations in the target binary, enabling the bypassing of complex branch constraints during fuzzing.

The option combination generation module begins by identifying potential command-line parameter processing functions from the source code. Then, an LLM simulates the debugging process for the target binary by comparing the expected function call sequence with the actual execution sequence. Based on the analysis, the LLM updates the command-line parameters and re-executes the program. This process iterates until the LLM identifies a program option combination that reaches the target code or determines that execution cannot proceed further.

The trace-guided fuzzing module utilizes historical traces to efficiently fuzz the target software and verify vulnerabilities. Alongside standard fuzzing mutation strategies, we introduce a Key-Bytes Guided Mutation strategy, which leverages key-byte traces to mutate seeds. Additionally, we propose a Nested Simulated Annealing algorithm. Specifically, we record key jump points along vulnerability paths from function-level traces to construct a state machine. For each test case, energy is allocated based on its execution state and the most desirable state to explore. This approach aims to prioritize exploration of the most promising state path while minimizing the risk of overlooking other potential paths.

Finally, the process yields three possible outcomes: the vulnerability is triggered, the reused code is reached but the vulnerability is not triggered, or the reused code is not reached. In the first case, the propagated vulnerability is confirmed. For the latter two cases, we conclude that the vulnerability is either not triggerable or is challenging to trigger in the target software, requiring further manual verification. Compared to existing methods, while TransferFuzz-Pro may face challenges in analyzing target binaries without crashes, it provides strong evidence of vulnerability propagation when the PoC successfully triggers the vulnerability. Experimental results further demonstrate the effectiveness of TransferFuzz-Pro in verifying vulnerabilities.

B. Historical Trace Extraction

In this section, we focus on extracting historical traces from the basic binary. We identify two types of runtime information that significantly aid in fuzzing the target binary: the function call sequence (FCS) of the basic binary and the key bytes within the PoC.

1) Function-level Traces Extraction: Our first objective is to extract the function call sequence (FCS) from the basic binary.

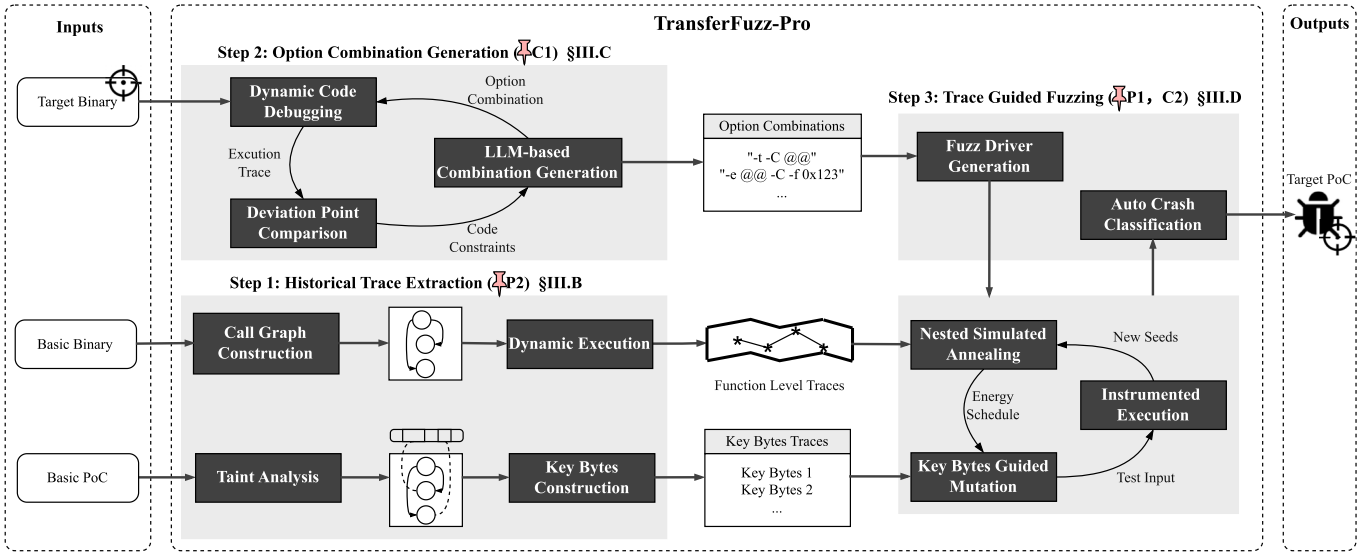


Fig. 4. The workflow of TransferFuzz-Pro. Transferuzz-Pro generates a PoC for the input target binary to verify whether the vulnerability affects the target binary.

Since the target and basic binaries often share reused code, vulnerabilities can typically be triggered along the same execution path. Thus, we aim to collect the vulnerability-triggering path from the basic binary to guide fuzzing in the target binary. To extract the FCS, we employ two methods: executing the PoC directly on the basic binary to record runtime information, and applying generic directed fuzzing on the basic binary to generate new PoCs and capture their runtime information.

Directly Execute PoC: For basic binaries with known vulnerability information and available PoCs, we directly execute the PoC to collect runtime information. This approach is particularly useful for vulnerabilities involving complex logic that are difficult to trigger through generic fuzzing. Our goal is to extract the function call sequence (FCS) that leads to the vulnerability, enabling the target binary to follow these sequences during fuzzing. This eliminates irrelevant paths and improves the precision and efficiency of the fuzzing process. To achieve this, we execute the basic binary and log the function call stack at the point of the crash.

Fuzzing the Basic Binary: For basic binaries without PoCs, or to expand the set of paths for those with existing PoCs, generic directed fuzzing can be used to generate additional PoCs. Relying on a single vulnerability-triggering path may limit the function-level traces extracted by TransferFuzz-Pro. In some cases, if the collected paths are not the easiest to trigger, they may hinder the fuzzing process for the target binary. Therefore, it is essential to gather a diverse set of paths for the basic binary through fuzzing beforehand. This process is performed offline and does not impact the runtime efficiency of TransferFuzz-Pro, as it only needs to be executed once for the basic binary to identify all potential propagation paths for the target binary. To achieve this, we use the state-of-the-art directed fuzzing tool SelectFuzz, which is designed for crash reproduction. By marking the source code line associated with the vulnerability, the fuzzer calculates the distance between the current test case and the target code, guiding seed mutations toward the vulnerability.

The motivation behind this approach lies in the observation that the basic binary and target binary often share the same vulnerability-triggering path within the reused code due to their shared codebase. Consequently, vulnerability paths that are easier to reach during fuzzing in the basic binary are also likely to be easier to reach in the target binary.

FCG Alignment: To apply the basic binary's function-level traces to the target binary, we need to align their functions. This is necessary because minor changes are often made to the reused code in the target binary. By performing function matching between the basic and target binaries, we can ensure that the function-level traces are compatible with the target binary.

The simplest method for function alignment is by directly using function names, which works if the target binary is not stripped. However, in practical scenarios where binaries may be stripped to remove function names, Binary Code Similarity Detection (BCSD) methods can be employed for function matching. In our evaluation, we used LibAM [5] to perform function matching in cases where function names were missing. Alternatively, other BCSD methods, such as those proposed in [26], [27], can also be used for this purpose.

After function matching, we extract a sub-path from the historical function call sequence of the basic binary. The original path connects the main function of the basic binary to the vulnerability function, while the sub-path corresponds to a segment of this path within the target binary, starting from a node in the original path and reaching the vulnerability function. This sub-path is recorded and used to guide fuzzing in the target binary directly.

2) Key-bytes Traces Extraction: In addition to function call sequences, we observed that certain bytes in the PoC can influence key condition variables in the code. Extracting these key bytes from the basic binary can significantly aid the fuzzing

Algorithm 1: Key Bytes Extraction Algorithm

Input: binaryExecutable B, reusedFunctionList F, proofOfConceptFile P
Output: accessedBytes A

Function `ExtractKeyBytes`(B, F, P)**:**
  $PBytes \leftarrow$ ReadBytes(P);
  $BImages \leftarrow$ LoadImages(B);
  foreach $image \in BImages$ **do**
    foreach $func \in F$ **do**
      if $func$ *is in* $image$ **then**
        InstrumentEntryAndExit($func$);
  $A \leftarrow \emptyset$;
  for *each instruction* I *in* B **do**
    if I *is MemoryAccess* ***and*** *IsInFunctionList(I, F)* **then**
      $address \leftarrow$ GetAccessAddress(I);
      if $address$ *corresponds to* $PBytes$ **then**
        $byte \leftarrow$ GetByteAt($address$, $PBytes$);
        $A \leftarrow A \cup \{(address, byte)\}$;
  return A;

process of the target binary. While this concept is similar to OCTOPOCs [28], it is important to note that OCTOPOCs simply extract key bytes from the PoC and splice them together to create a new PoC, which often fails. Instead, we propose using these key bytes as a dictionary to guide the mutation process for the target binary.

We implement this approach using taint analysis techniques. Specifically, the PoC for the basic binary is marked as tainted, and during execution, we monitor whether the conditional statement variables in the reused code are derived from the PoC. We then compare the bytes accessed by the reused code with those in the PoC to extract the bytes that are directly passed unchanged from the PoC, designating them as key-byte traces. The detailed process is outlined in Algorithm 1. During fuzzing of the target binary, inserting these tainted bytes at specific locations simplifies satisfying the corresponding conditional statements in the reused code.

C. Option Combination Generation

In this section, we aim to generate option combinations for the target binary, enabling the automatic creation of a fuzz driver for the subsequent fuzzing process. As shown in Fig. 5, we first identify the option processing function within the target binary. The option processing code, along with each debugger execution result, is then provided to the LLM to generate the required option combinations.

1) Locating Option Processing Function: Option processing functions typically exhibit distinct structural features, such as the use of switch statements or multiple if branches that execute different code paths based on specific option strings (e.g., "-v", "-t"). These structures are easily identifiable by LLMs. Additionally, option processing code is usually located directly

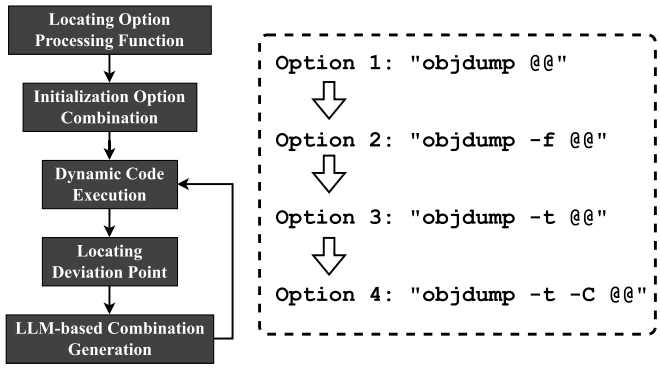


Fig. 5. LLM driven code debugging process.

in the main function or invoked as a separate function. Even when not directly called by main, it can typically be reached within a few function hops.

To extract option processing code, we use an LLM to analyze the main function and identify potential option processing functions. If a candidate function is detected, the LLM recursively analyzes it to determine if it contains option processing code. This process continues iteratively until the option processing code is located or a maximum recursion depth is reached. Based on our experimental dataset, we set the recursion depth to 3, which has proven sufficient. However, in practical usage, this depth can be increased for more complex cases. The prompt template used for this process is detailed in Table I.

2) Generating Option Combination: Once the option-processing code is identified, we can utilize LLMs for code debugging. Option-processing code typically sets flag variables or invokes specific functions based on the provided options. As illustrated in Fig. 2, function calls directly alter the execution path, while flag variables influence conditional branch statements within functions, guiding the program along specific paths. Thus, it is sufficient to extract the branch-related code and the option-processing code that modifies the function's execution path. Leveraging the semantic analysis capabilities of LLMs, we can then determine the options required to execute a specific path. The prompt template for this process is detailed in Table I.

We begin by performing static analysis to extract all expected execution paths that may run into the function call sequence in the historical trace. Next, we initialize an empty set of options and use a debugger to execute the target binary. The execution path of the target binary is recorded and compared with the expected path to identify deviation points. The deviation point code and option processing code are then provided to the LLM, which analyzes the code semantics to extract constraints and corresponding options needed to progress along the expected path. The options are updated, and the target binary is re-executed. This process is repeated until either the vulnerability-related code is reached along an expected path or the program execution ceases to progress.

In addition to leveraging LLM's code semantic analysis capabilities, certain binaries require numeric inputs as parameters. For instance, *addr2line* requires an address, such as *0x123*, as

TABLE I
PROMPT FOR EACH TASK IN OPTION COMBINATION GENERATION MODULE

Task	Prompt Template
Locating Option Processing Function	Please extract the code for processing options from `<function_name>`. If the processing code is in the `<function_name>`, return the corresponding code in json format; if the processing code is a function call, return the function name in json format. Output the final result only as a dictionary. The code of `<function_name>`: `<function_body>`
Option Combination Generation	The following is the execution trace of `<command>` from Valgrind. With current command line parameters, `<function_name1>` does not call `<function_name2>`. Analyze what options should be added to run to `<function_name2>`. Please only output a json dictionary containing the executed command. The execution trace: `<execution_trace>`. The options processing code: `<option_processing_code>`

part of its options. While LLMs can often analyze parameter-related code successfully, they occasionally fail to directly locate code associated with such special parameters. However, due to their extensive training on diverse datasets, LLMs possess inherent knowledge that can address these cases. For example, in the case of *addr2line*, the LLM autonomously generated a valid 16-bit address parameter, satisfying the expected path constraints. In our evaluation, all the numbers with loose restrictions were successfully generated. While this approach may fail in more complex scenarios or for less commonly used binaries, such cases are rare and are left for future research.

D. Trace Guided Fuzzing

This section focuses on transferring the extracted historical traces to the target binary and utilizing them to efficiently verify the propagated vulnerability code in the target binary through fuzzing.

1) Nested Simulated Annealing: Unlike directed fuzzing, which focuses on a single target line of code, TransferFuzz-Pro utilizes function call sequences from historical traces to allocate more energy to seeds exploring vulnerability paths and less energy to those on unrelated paths.

To achieve this, a unique state machine is created for each execution path, recording every function encountered along the path during fuzzing. Each state machine corresponds to an individual historical trace, simplifying the management of multiple traces. In these state machines, each state represents a function node, forming a sequential path from the entry function of the reused code to the function containing the vulnerability. Energy allocation for test cases is determined by comparing their current states with the most recently reached state in the corresponding state machine, prioritizing paths that are most relevant to the vulnerability.

The goal is to focus energy on advancing along the most recently identified states while avoiding over-prioritizing the deepest path at the expense of other potentially vulnerable paths. To strike this balance, we employ the simulated annealing algorithm, as described in AFLGo. Simulated Annealing (SA) [29], inspired by the annealing process in metallurgy, involves controlled heating and cooling to optimize a system by reducing defects. In AFLGo, the algorithm transitions at a designated point, tx, from an exploratory phase to a more focused exploitation phase. After reaching tx, the algorithm mimics a classical gradient descent approach, optimizing the search greedily. Before this point, it ensures diverse exploration of alternative paths, maintaining a comprehensive assessment of potential vulnerabilities.

Algorithm 2: Nested Simulated Annealing Algorithm

Input: current state cur, states list $states$, start time list $time$, current time t
Output: energy E

Function `CalcEnergy`(cur, $states$, $time$, t):
  if cur *is new* **then**
    NewState(t, $states$, $time$);
    return 0.5;
  $idx \leftarrow$ Index of cur in $states$;
  $start \leftarrow time[idx]$;
  $temp \leftarrow$ `SimulatedAnnealing`($start$, t);
  $E \leftarrow 1 - 0.5 \times temp$;
  for $i = idx + 1$ **to** *length of* $states$ **do**
    $newStart \leftarrow time[i]$;
    $newTemp \leftarrow$ `SimulatedAnnealing`($newStart$, t);
    $E \leftarrow E \times 0.5 \times newTemp$;
  return E;
;
Function `SimulatedAnnealing`($start$, t):
  $T \leftarrow$ Cooling($t, start$);
  return T;

Unlike AFLGo's single Markov chain model, we model each execution path as an independent state machine, where each node represents a function along the path. Each state machine maintains a timeline that records when each state is first encountered. This design transforms energy scheduling into a probabilistic task, distributing energy dynamically among states. When a new state emerges, the initial energy allocation between the new and existing states is set to 0.5. Over time, this ratio is adjusted using the simulated annealing algorithm, progressively increasing the energy assigned to newer states while decreasing it for older ones. This approach allows each state machine to independently manage its simulated annealing timeline, ensuring the system collectively converges toward the most relevant states. The pseudo-code for this algorithm is presented in Algorithm 2.

In detail, for each input seed, the energy is calculated based on the state reached during the execution. The calculation of

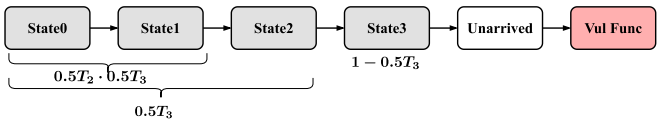


Fig. 6. State machine diagram in NSA algorithm.

energy for every state is defined as Equation 1:

$$E_j = \begin{cases} \prod_{i>j} 0.5T_i & \text{if } j = 0, \\ (1 - 0.5T_j) \cdot \prod_{i>j} 0.5T_i & \text{if } j > 0. \end{cases} \quad (1)$$

Within this context, E_j denotes the energy of the $j\ th$ state, while T_j represents the temperature parameter [15] for the $j\ th$ state, calculated using the simulated annealing algorithm.

The detailed procedure of the nested simulated annealing algorithm is illustrated using the example in Fig. 6. Initially, TransferFuzz-Pro operates similarly to a generic directed fuzzing tool until it reaches the reused code area. During this phase, energy scheduling follows a basic simulated annealing strategy [15]. Once *state0* of the state machine is encountered, the first simulated annealing algorithm is triggered. This algorithm allocates half of the energy (0.5) to seeds that have reached *state0*, while the remaining 0.5 is distributed to seeds that have not. Over time, the energy assigned to *state0* increases to 1, while the energy for seeds failing to reach *state0* diminishes to zero.

When a new state, *state1*, is first reached, a second simulated annealing algorithm is activated. This algorithm assigns half of the energy (0.5) to seeds at *state1*, while the remaining 0.5 is shared among seeds stuck at *state0* or earlier states, adhering to the principles of the first algorithm. Each time a new state is reached, a new simulated annealing algorithm is triggered.

By employing this nested simulated annealing approach, the energy allocated to seeds unrelated to the vulnerability paths in the historical trace decays rapidly. This ensures that mutations increasingly focus on driving the state machine forward along paths relevant to the vulnerability.

2) Key Bytes Guided Mutation: TransferFuzz-Pro builds upon the mutation strategy of traditional directed fuzzing by introducing significant enhancements. Its key innovation lies in leveraging key bytes extracted from the binary's historical traces to guide test case generation. Specifically, each unique sequence of consecutively accessed bytes in memory is added as an entry in the dictionary. By systematically incorporating these key bytes into the dictionary and applying them during each mutation phase, TransferFuzz-Pro effectively bypasses challenging branch constraints, leading to a measurable improvement in fuzzing efficiency, as confirmed by our experimental results.

While the Key-Bytes Guided Mutation (KBGM) algorithm shares similarities with the taint analysis techniques employed in tools like AFL++ [30], WindRanger [18], and REDQUEEN [31], it is notably more robust. KBGM extracts critical information from another binary and is capable of addressing branches with complex conditions. This ability enables it to handle sophisticated vulnerabilities, as demonstrated in our evaluation.

IV. Implementation

We implemented the TransferFuzz-Pro prototype with over 3,000 lines of Python and C code. TransferFuzz-Pro is a generalized, historical trace-based fuzzing framework that can build on any directed fuzzing technique. For our implementation, we selected the latest directed fuzzing framework, SelectFuzz [21].

Historical Trace Extraction: To extract function call sequences from the basic binary, we employed GDB [32] for dynamic debugging, automating the extraction process through custom Python scripts. These scripts capture the function call stacks after crashes. For key byte extraction, we utilized PIN [33] to perform taint analysis, identifying whether the tainted bytes directly originate from the PoC.

Option Combination Generation: To identify execution paths that assist LLMs in code debugging, we used debuggers such as GDB and Valgrind. Valgrind was our primary choice for its speed and fine-grained analysis, while GDB was used for binaries that Valgrind could not handle. During execution, we recorded the execution trace of the target binary. Additionally, we leveraged LLM APIs without retraining or supervised fine-tuning (SFT). After evaluating cost and performance, we selected the cost-efficient Claude3.5-Sonnet as the base LLM, as it adequately handled this relatively simple task. More powerful models, such as GPT-4o or o1-preview, provided no significant performance gains. In our experiments, the cost per case was approximately $0.3.

Trace-Guided Fuzzing: We used IDA Pro to reverse engineer the target binary and inserted assembly instrumentation into each block, similar to AFL [30]. For the Key-Bytes Guided Mutation strategy, we maintained the extracted key bytes in a dictionary and passed them as arguments to the fuzzer. For the Nested Simulated Annealing algorithm, we stored the function call sequences from the historical trace as specific indices in a global array that tracks test case coverage edges during fuzzing. After executing a new coverage edge, we determined the current test case's state by checking whether the edge matched the indices in the global array. This information was then used to update the global state machine.

By implementing this approach, TransferFuzz-Pro incurs negligible space and time overhead, ensuring its execution speed matches that of generic directed fuzzing. Additionally, we developed a shell script to automate verification completion by analyzing crash addresses via GDB. Finally, we implemented two variants of TransferFuzz-Pro: TF-Pro (source code instrumentation) for targets with available source code. TF-Pro$_{bin}$ (binary instrumentation) for targets without source code.

V. Evaluation

In this section, we evaluate TransferFuzz-Pro alongside existing state-of-the-art (SOTA) methods. Our analysis includes a detailed investigation of the limitations of existing approaches, identifying the causes of their shortcomings. Prior to this, we outline our experimental setup, the dataset used, and the baseline methods for comparison.

The experiments are designed to address the following research questions:

RQ1: How does TransferFuzz-Pro automatically generate option combinations?

RQ2: How does TransferFuzz-Pro perform in the Vulnerability Verification of Reused Code task?

RQ3: How efficiently does TransferFuzz-Pro trigger vulnerabilities in target binaries?

RQ4: How does each component of TransferFuzz-Pro contribute to its overall performance?

RQ5: Can TransferFuzz-Pro generate PoCs for various affected real-world Linux distributions?

The experiments were conducted on an Ubuntu 22.04 system, running on an Intel Xeon CPU with 128 cores clocked at 3.0GHz, with hyperthreading enabled. Each fuzzer operated within an isolated Docker container to ensure reproducibility and isolation.

To assess the performance of TransferFuzz-Pro and the baseline methods, we employed the following evaluation metrics:

To answer RQ2, we use Precision and Recall to evaluate whether TransferFuzz-Pro can verify more vulnerabilities and reduce false positives compared to existing methods. Specifically, TP represents the number of correctly verified propagated vulnerabilities, while TN denotes the number of non-propagated vulnerabilities that are correctly detected as non-propagated. Conversely, FN indicates the number of non-propagated vulnerabilities that are incorrectly verified while FP presents the number of propagated vulnerabilities that are incorrectly detected as propagated. Moreover, the equations of Precision and Recall are as follows:

$$Precision = \frac{TP}{TP + FP} \tag{2}$$

$$Recall = \frac{TP}{TP + FN} \tag{3}$$

To answer the RQ3 and RQ4, we use the Time-to-Exposure (TTE) to evaluate the efficiency. Similar to previous work [18], each method was repeated 10 times to take the average μTTE with a time budget of 12 hours. We calculate the sum of μTTE for vulnerabilities successfully triggered 10 times and evaluate the percent TrasferFuzz speed up.

TTE: Time-to-Exposure (TTE) is used to measure the time used by the fuzzer to trigger the vulnerability.

A. Dataset

We conducted code reuse detection on datasets from AFLGo [15], SelectFuzz [21], and DAFL [20], extracting binaries containing propagated vulnerable functions. Although OCTOPOCs [28] did not release its dataset, we located six of its referenced PDF processing software online. In total, we collected 15 CVE vulnerabilities with code reuse and 76 related binaries, as summarized in Table II.

Leveraging LibAM, a state-of-the-art code reuse detection tool, we identified 146 potential vulnerabilities by detecting the presence of vulnerable code. After manually filtering out 108 false positives, we found that the initial 15 CVE vulnerabilities had broader implications, affecting additional software and expanding to 53 distinct vulnerabilities. This dataset highlights the effectiveness of TransferFuzz-Pro in vulnerability validation.

B. Compared Methods

In this evaluation, we compare TransferFuzz-Pro with several existing methods. The comparison approaches are outlined as follows:

LibAM [5]: A state-of-the-art (SOTA) third-party library code reuse detection method, representing code reuse-based approaches.

OCTOPOCs [28]: The only method with a concept similar to ours, leveraging symbolic execution and taint analysis for verifying propagating vulnerable code.

AFLGo [15]: A foundational and classic directed fuzzing technique, serving as an early benchmark in the field.

Windranger [18]: A directed fuzzing approach that prioritizes deviation basic blocks for improved efficiency.

SelectFuzz [21]: An advanced directed fuzzing method that incorporates selective instrumentation to optimize performance.

DAFL [20]: A modern directed fuzzing technique that utilizes data flow distance metrics to guide fuzzing.

C. Answer to RQ1: Accuracy of Option Combination Generation

In this section, we evaluate whether TransferFuzz-Pro can accurately generate option combinations for target binaries to reach vulnerable code. To ensure a comprehensive assessment, we included not only the 53 vulnerabilities listed in Table II but also additional vulnerabilities from the dataset that do not involve code reuse but require specific command-line parameters to trigger. In total, 60 vulnerabilities were used to calculate the accuracy of option combination generation.

There is no existing work to achieve this goal. For comparison, we define two baselines. The first, LLM_o, uses the inherent knowledge of the large language model to directly generate option combinations for the target binary. The second, LLM_p, inputs only the option processing code into the LLM without incorporating debugging. These baselines allow us to separately evaluate the impact of the model's inherent knowledge and its code comprehension capabilities, highlighting the necessity of our code debugging techniques. For each (binary, vulnerability) combination, we determine whether the generated option combination is correct and calculate the precision using Formula 2.

Table III provides examples of cases requiring specific option combinations. To better reflect real-world scenarios, we retained vulnerabilities that do not require specific option combinations. TransferFuzz-Pro successfully generated ideal option combinations for 85.0% of target binaries, significantly outperforming LLM_o and LLM_p, which achieved 45.0% and 50.0%, respectively. The maximum number of iterations for TransferFuzz-Pro was 7. For binaries like *addr2line* that require additional address inputs, the LLM either analyzed option requirements from the code or generated the necessary options using its inherent knowledge. For *objdump*, the LLM identified more option combinations capable of triggering vulnerabilities compared to prior manual analysis.

In contrast, the other two methods struggle with cases requiring specific options or complex combinations. They typically

TABLE II
DATASET SCALE

Vul	Source	Type	CrashFunc	Code Reuse	Propogated	Propogated Binaries
CVE-2016-10095	tiffsplit	CWE-119	_TIFFVGetField	24	2	thumbnail, tiffcmp
CVE-2016-5318	thumbnail	CWE-119	_TIFFVGetField	24	2	tiffsplit, tiffcmp
CVE-2016-4487	cxxfilt	CWE-416	register_Btype	11	3	objdump, nm-new, addr2line
CVE-2016-4489	cxxfilt	CWE-190	string_appendn	11	3	objdump, nm-new, addr2line
CVE-2016-4490	cxxfilt	CWE-190	d_unqualified_name	11	3	objdump, nm-new, addr2line
CVE-2016-4491	cxxfilt	CWE-119	d_print_comp_inner	11	3	objdump, nm-new, addr2line
CVE-2016-4492	cxxfilt	CWE-119	do_type	11	3	objdump, nm-new, addr2line
CVE-2016-6131	cxxfilt	CWE-20	demangle_class_name	11	3	objdump, nm-new, addr2line
CVE-2017-7303	strip	CWE-125	section_match	11	1	objcopy
CVE-2017-11733	swftocxx	CWE-125	stackswap	6	4	swftophp, swftoperl, swftopython, swftotcl
CVE-2018-8807	swftophp	CWE-416	getString	6	4	swftocxx, swftoperl, swftopython, swftotcl
CVE-2018-8962	swftophp	CWE-416	getName	6	4	swftocxx, swftoperl, swftopython, swftotcl
CVE-2017-18267	Poppler	CWE-476	FoFiType1C::getOp	1	1	xpdf
CVE-2018-11102	libav	CWE-119	mov_probe	1	1	ffmpeg
CVE-2018-20330	libjpeg-turbo	CWE-190	__memcpy_avx_unaligned	1	1	mozjpeg

TABLE III
ACCURACY IN OPTION COMBINATION GENERATION TASK

Needed Option Combination	LLM_o	LLM_p	TF	TF-Pro
cxxfilt < @@	✘	✘	✘	✘
objdump -t -C @@	✘	✘	✘	✔
nm-new -C @@	✘	✔	✘	✔
addr2line -e @@ -C -f 0x123	✘	✘	✘	✔
strip -o /dev/null @@	✘	✘	✘	✔
avconv -y -i @@	✔	✔	✘	✔
ffmpeg -y -i @@	✔	✔	✘	✔
tjbench @@ 90	✘	✘	✘	✘
djpeg -colors 256 -bmp @@	✘	✘	✘	✔
tiffcrop -i @@ /dev/null	✘	✘	✘	✔
gif2tga -i @@	✘	✔	✘	✔
readelf -w @@	✘	✔	✘	✔
split -C 1024 @@	✘	✘	✘	✘
wvunpack -m @@ -o output	✔	✔	✘	✔
xmllint –recover –sax1 –sax @@	✘	✘	✘	✔

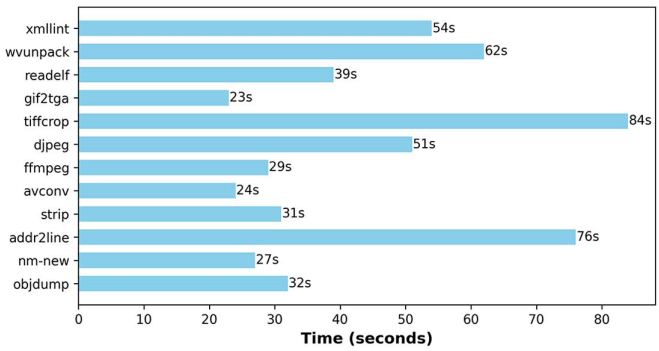


Fig. 7. Time cost of option generation task.

rely on randomly generating a small number of options, often failing to produce effective combinations. Besides, TransferFuzz (TF) requires manual analysis of the Options required for Fuzzing for all test items. As shown in Fig. 7, TF-Pro adds less than 90 seconds for automated option generation. In contrast, an expert binary analyst (5+ years of experience) requires more than 20 minutes per case for manual analysis in TransferFuzz, highlighting TF-Pro's substantial efficiency improvement.

In addition, we analyzed the reasons for the failure cases in detail. On one hand, TransferFuzz-Pro may fail to generate options that require complex calculations. For example, if a project like *split* needs a specific parameter value (e.g., exactly 1024), TransferFuzz-Pro may be unable to produce the complete option combination. Nonetheless, it can still generate the required "-C" option except for the precise number, effectively reducing manual effort. On the other hand, TransferFuzz-Pro encounters difficulty when option selection lacks clear semantic cues in the code. While LLMs can leverage world knowledge to handle common projects, they may fail with niche projects where such information is not available; even human analysts may struggle in these cases. In future work, incorporating project documentation or manuals as additional input to the LLM could further improve option generation accuracy.

Answering RQ1: TransferFuzz-Pro can automatically generate 85% of Options within 90 seconds, demonstrating the effectiveness of the LLM-driven code debugging approach.

D. Answer to RQ2: Accuracy of Verifying Propagated Vulnerability Code

In this section, we evaluate whether TransferFuzz-Pro can accurately validate vulnerabilities in propagated code compared to existing methodologies.

As shown in Table IV, TransferFuzz-Pro achieves a precision of 1.0 and a recall of 1.0, successfully validating all 38 propagated vulnerability samples. This demonstrates that TransferFuzz-Pro provides more accurate and comprehensive vulnerability validation than existing methods.

While LibAM also achieves a recall of 1.0, it focuses solely on code similarity detection without confirming whether the identified vulnerabilities are triggerable. Consequently, it suffers from a significantly low precision of 0.260.

OCTOPOCs is the first approach designed to transfer Proof-of-Concepts (PoCs) from basic binaries to target binaries. However, it faces challenges such as path explosion and a strict

TABLE IV
ACCURACY IN THE VULNERABILITY VERIFICATION TASK

Model	TP	FP	TN	FN	Precision	Recall
LibAM	38	108	0	0	0.260	1.000
OCTOPOCs	15	0	108	23	1.000	0.395
AFLGo	27	0	108	11	1.000	0.711
Windranger	29	0	108	10	1.000	0.763
SelectFuzz	29	0	108	10	1.000	0.763
DAFL	20	0	108	18	1.000	0.526
TF / TF-Pro	**38**	**0**	**108**	**0**	**1.000**	**1.000**

requirement for target binaries to share the same input format as the basic binaries. As a result, OCTOPOCs is limited to cases where the PoCs of source and target binaries are consistent, which accounts for less than half of our dataset.

Other directed fuzzing methods exhibit high precision but suffer from significantly lower recall rates compared to TransferFuzz-Pro. This limitation arises from their inability to validate vulnerabilities with complex logic, particularly those that are difficult to trigger. TransferFuzz-Pro addresses this challenge by leveraging historical traces extracted from PoCs in basic binaries, enabling it to validate even the most intricate vulnerabilities.

Answering RQ2: TransferFuzz-Pro demonstrates the capability to accurately validate vulnerabilities, achieving the highest precision and recall. TransferFuzz-Pro successfully validated 38 propagated vulnerability samples.

E. Answer to RQ3: Efficiency in Triggering Vulnerabilities

In this section, we assess the speed of TransferFuzz-Pro in comparison to existing directed fuzzing methods. Since only TransferFuzz-Pro automates option generation, Table V fairly compares only the fuzzing stage. Consistent with the approach in SelectFuzz [21], we used empty files or simple test files (e.g., basic ELF or PDF files) from the target project as initial seeds for all methods. While some PoCs from the base binary (e.g., from LibMing and OCTOPOCs) are directly applicable to target binaries, they represent only a small subset of our dataset. To ensure a fair comparison, we avoided shortcuts by not using PoCs from the base binaries, unlike the settings in RQ1. We also analyzed each sample to explain why TransferFuzz-Pro surpasses existing approaches in performance. Note that the performance evaluation in Table V only represents the time of the dynamic fuzzing stage. The time before fuzzing also includes the time of static analysis distance calculation and Option Generation time. Since TF and TF-Pro do not add additional calculations, the static analysis time is the same as Selectuzz, and the time of TF-Pro's Option Generation process is shown in Fig. 8.

As shown in Table V, TF/TF-Pro is the fastest method, achieving a speed improvement of 2.5x over the state-of-the-art SelectFuzz. Nearly all vulnerabilities are verified within 60 minutes, and over half are confirmed within 2 minutes.

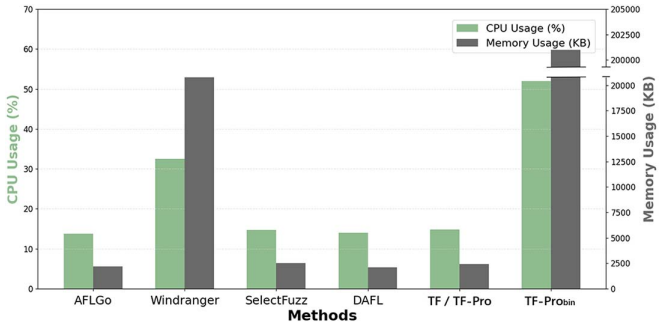


Fig. 8. CPU and memory usage. The green bars represent CPU usage, while the gray bars indicate memory usage (RES).

However, without leveraging PoCs from the base binary, none of the methods, including TransferFuzz-Pro, were able to generate PoCs for three vulnerabilities (CVE-2017-18267, CVE-2018-11102, and CVE-2018-20330). This limitation is due to the prototype nature of existing methods, which require additional engineering efforts, such as in-process instrumentation, to adapt to a broader range of software. We plan to enhance existing fuzzers in future work. TF-Pro_bin verification speed is slightly slower but still exceeds existing methods.

Other methods often require several hours to trigger vulnerabilities and fail to do so for more complex cases, such as CVE-2016-4491, CVE-2016-4492, and CVE-2016-6131. In contrast, the KBGM algorithm in TransferFuzz-Pro reduces the time required to trigger vulnerabilities from several hours to just a few minutes (e.g., Vul-10). Even for vulnerabilities without complex branching, TransferFuzz-Pro significantly improves the performance of existing methods (e.g., Vul-28 to Vul-35) through the NSA module. A detailed discussion of these results is provided in Section V-F.

TF-Pro matches TransferFuzz's performance (no fuzzing modifications), while TF-Pro$_{bin}$ incurs minor slowdowns due to binary instrumentation. Despite this, TF-Pro$_{bin}$ outperforms existing methods and uniquely supports binary-only verification, addressing a limitation of TransferFuzz.

We also evaluated the resource consumption of TransferFuzz-Pro compared to other tools by measuring maximum CPU usage and memory usage (RES) during the fuzzing process for each vulnerability. As illustrated in Fig. 8, the resource consumption of TF/TF-Pro is comparable to AFLGo, SelectFuzz, and DAFL. However, Windranger exhibited a significant increase in memory usage due to its continuous reliance on taint analysis throughout the fuzzing process. Despite this, the resource consumption of all methods, including TransferFuzz-Pro, remains manageable and does not hinder their applicability for vulnerability verification. TF-Pro preserves the efficiency of TransferFuzz, as it does not modify the fuzzing process. For TF-Pro$_{bin}$, the increased resource consumption (primarily due to QEMU using over 200MB of memory) is reasonable and consistent with other binary instrumentation-based fuzzing tools.

Answering RQ3: TransferFuzz-Pro consistently outperforms existing methods in terms of both speed and the number of vulnerabilities verified.

TABLE V
THE NUMBER OF RUNS AND EFFICIENCY FOR EACH VULNERABILITY

No.	Vulnerability	AFLGo	Windranger	SelectFuzz	DAFL	TF / TF-Pro	TF-Pro$_{bin}$
1	thumbnail-2016-10095	10m35s (10)	3m27s (10)	2m13s (10)	3m38s (10)	**0m56s** (10)	1m8s (10)
2	tiffcmp-2016-10095	8m24s (10)	5m16s (10)	2m32s (10)	4m17s (10)	**1m49s** (10)	2m12s (10)
3	tiffsplit-2016-5318	7m27 (10)	6m5s (10)	1m24s (10)	2m53s (10)	**1m28s** (10)	1m47s (10)
4	tiffcmp-2016-5318	7m12s (10)	4m39s (10)	3m5s (10)	4m24s (10)	**1m38s** (10)	2m09s (10)
5	objdump-2016-4487	27m23s (10)	5m32s (10)	14m17s (10)	9m54s (10)	**0m5s** (10)	0m6s (10)
6	nm-new-2016-4487	35m31s (10)	6m12s (10)	12m32s (10)	13m56s (10)	**0m40s** (10)	0m52s (10)
7	addr2line-2016-4487	1h5m45s (10)	1h14m23s (10)	43m23s (10)	1h3m24s (10)	**0m38s** (10)	0m49s (10)
8	objdump-2016-4489	48m5s (10)	7m16s (10)	17m23s (10)	13m45s (10)	**0m4s** (10)	0m7s (10)
9	nm-new-2016-4489	34m51s (10)	3m42s (10)	13m51s (10)	9m13s (10)	**0m27s** (10)	0m43s (10)
10	addr2line-2016-4489	1h47m15s (10)	1h52m0s (10)	1h3m25s (10)	1h24m26s (10)	**0m51s** (10)	1m7s (10)
11	objdump-2016-4490	21m45s (10)	4m14s (10)	13m26s (10)	9m42s (10)	**0m4s** (10)	0m9s (10)
12	nm-new-2016-4490	35m23s (10)	11m24s (10)	9m52s (10)	13m26s (10)	**1m3s** (10)	1m23s (10)
13	addr2line-2016-4490	2h4m21s (10)	1h3m25s (10)	58m25s (10)	42m52s (10)	**0m53s** (10)	1m5s (10)
14	objdump-2016-4491	N.A. (0)	N.A. (0)	N.A. (0)	N.A. (0)	**3m37s** (10)	4m41s (10)
15	nm-new-2016-4491	N.A. (0)	N.A. (0)	N.A. (0)	N.A. (0)	**4m15s** (10)	5m8s (10)
16	addr2line-2016-4491	N.A. (0)	N.A. (0)	N.A. (0)	N.A. (0)	**7m14s** (10)	9m21s (10)
17	objdump-2016-4492	56m24s (10)	46m11s (10)	23m15s (10)	35m1s (10)	**0m32s** (10)	0m39s (10)
18	nm-new-2016-4492	1h2m42s (10)	58m15s (10)	44m35s (10)	39m25s (10)	**0m40s** (10)	0m49s (10)
19	addr2line-2016-4492	N.A. (0)	N.A. (0)	2h46m (3)	3h41m58s (2)	**0m17s** (10)	0m21s (10)
20	objdump-2016-6131	N.A. (0)	N.A. (0)	N.A. (0)	N.A. (0)	**0m4s** (10)	0m6s (10)
21	nm-new-2016-6131	N.A. (0)	N.A. (0)	N.A. (0)	N.A. (0)	**0m56s** (10)	1m18s (10)
22	addr2line-2016-6131	N.A. (0)	N.A. (0)	N.A. (0)	N.A. (0)	**0m32s** (10)	0m42s (10)
23	objcopy-2017-7303	N.A. (0)	2h33m (3)	1h57m12s (5)	N.A. (0)	**1h30m2s** (6)	1h52m (6)
24	swftophp-2017-11733	5m16s (10)	1m59s (10)	0m32s (10)	1m23s (10)	**0m6s** (10)	0m8s (10)
25	swftoperl-2017-11733	4m25s (10)	1m42s (10)	**0m34s** (10)	2m0s (10)	0m52s (10)	1m3s (10)
26	swftopython-2017-11733	6m43s (10)	3m1s (10)	0m46s (10)	1m25s (10)	**0m28s** (10)	0m35s (10)
27	swftotcl-2017-11733	3m25s (10)	2m48s (10)	1m22s (10)	2m34s (10)	**0m25s** (10)	0m30s (10)
28	swftocxx-2018-8807	2h24m17s (10)	40m45s (10)	32m14s (10)	N.A. (0)	**20m2s** (10)	24m35s (10)
29	swftoperl-2018-8807	2h4m17s (10)	51m32s (10)	19m15s (10)	N.A. (0)	**9m21s** (10)	12m18s (10)
30	swftopython-2018-8807	2h37m42s (10)	47m13s (10)	26m41s (10)	N.A. (0)	**17m2s** (10)	21m45s (10)
31	swftotcl-2018-8807	2h8m32s (10)	54m21s (10)	29m1s (10)	N.A. (0)	**22m49s** (10)	32m17s (10)
32	swftocxx-2018-8962	3h43m54s (10)	1h13m45s (10)	45m15s (10)	N.A. (0)	**33m42s** (10)	41m13s (10)
33	swftoperl-2018-8962	3h24m16s (10)	56m34s (10)	33m25s (10)	N.A. (0)	**29m15s** (10)	35m9s (10)
34	swftopython-2018-8962	3h52m1s (10)	1h3m52s (10)	58m21s (10)	N.A. (0)	**55m20s** (10)	1h17m3s (10)
35	swftotcl-2018-8962	3h35m15s (10)	52m16s (10)	1h1m15s (10)	N.A. (0)	**46m16s** (10)	56m7s (10)
μTTE inc (all binaries)		**+850%**	**+344%**	**+248%**	**+2620%**		

Note: μTTE (RUNS) denote the average time and the number of successful validation runs (10 attempts per vulnerability).

F. Answer to RQ4: Impact of Different Components

In this section, we evaluate the effectiveness of each module within TransferFuzz-Pro. To do so, we performed ablation experiments by excluding the Key Bytes Guided Mutation strategy (referred to as No KBGM) and the Nested Simulated Annealing algorithm (referred to as No NSA).

In the absence of KBGM (No KBGM), TransferFuzz-Pro showed a significant drop in efficiency due to the lack of key-byte traces from taint analysis, which are critical for bypassing essential constraints. This performance degradation was particularly evident for vulnerabilities such as CVE-2016-4490, CVE-2016-4491, CVE-2016-4492, and CVE-2016-6131. However, the impact on CVE-2017-11733 was minimal, indicating that KBGM's effectiveness in extracting key bytes is not uniform across all cases. Despite this limitation, No KBGM still outperformed alternative methods in terms of speed.

For No NSA, TransferFuzz-Pro also experienced a decline in performance, though it was less severe compared to No KBGM. A consistent reduction in efficiency was observed, as the NSA module contributed to performance improvements across all tested vulnerabilities.

> **Answering RQ4:** KBGM and NSA both contribute to the performance of TransferFuzz-Pro.

G. Answer to RQ5: PoC for Linux Distributions Generation

In this section, we aim to detect and verify vulnerabilities in real-world Linux distributions. Notably, many Linux distributions include Binutils. Using TransferFuzz-Pro, we generate PoCs for the affected versions of these distributions. The results are summarized in Table VII.

TABLE VI
EFFICIENCY OF DIFFERENT MODULES OF TRANSFERFUZZ-PRO

CVE-ID	Targets	No KBGM	No NSA	TF / TF-Pro
2016-10095	thumbnail	1m59s	1m2s	**0m56s**
	tiffcmp	3m1s	2m24s	**1m49s**
2016-5318	tiffsplit	1m54s	1m31s	**1m28s**
	tiffcmp	2m6s	1m56s	**1m38s**
2016-4487	objdump	6m32s	0m9s	**0m5s**
	nm-new	7m45s	0m51s	**0m40s**
	addr2line	26m51s	0m56s	**0m38s**
2016-4489	objdump	14m2s	0m58s	**0m54s**
	nm-new	11m41s	0m34s	**0m27s**
	addr2line	47m26s	1m14s	**0m51s**
2016-4490	objdump	12m51s	0m4s	**0m4s**
	nm-new	9m3s	1m18s	**1m3s**
	addr2line	34m21s	1m21s	**0m53s**
2016-4491	objdump	N.A.	6m19s	**3m37s**
	nm-new	N.A.	6m43s	**4m15s**
	addr2line	N.A.	9m12s	**7m14s**
2016-4492	objdump	19m51s	**0m29s**	0m32s
	nm-new	25m32s	0m47s	**0m40s**
	addr2line	1h18m3s(5)	0m36s	**0m17s**
2016-6131	objdump	N.A.	0m5s	**0m4s**
	nm-new	N.A.	1m6s	**0m56s**
	addr2line	N.A.	0m47s	**0m32s**
2017-7303	objcopy	1h43m16s(7)	2h14m(4)	**1h30m**(6)
2017-11733	swftophp	18s	0m26s	**0m6s**
	swftoperl	0m58s	0m43s	**0m52s**
	swftopython	0m41s	0m35s	**0m28s**
	swftotcl	1m12s	0m37s	**0m25s**
2018-8807	swftocxx	**18m49s**	28m13s	20m2s
	swftoperl	9m52s	16m4s	**9m21s**
	swftopython	18m34s	21m15s	**17m2s**
	swftotcl	22m56s	23m35s	**22m49s**
2018-8962	swftocxx	37m25s	41m38s	**33m42s**
	swftoperl	31m25s	34m21s	**29m15s**
	swftopython	56m16	57m54s	**55m20s**
	swftotcl	51m46s	**43m19s**	46m16s

TABLE VII
REAL WORLD LINUX DISTRIBUTIONS EVALUATION

OS	Vulnerabilities
Debian 8	CVE-2016-4487, 4489, 4490, 4491, 6131, CVE-2017-7303
Debian 9	CVE-2016-4487, 4489, 4490, 4491, 6131, CVE-2017-7303
OpenWrt 16	CVE-2017-7303
Ubuntu 16	CVE-2016-4487, 4489, 4490, 4491, 6131, CVE-2017-7303
CentOS 7	CVE-2017-7303
Fedora 24	CVE-2016-4487, 4489, 4490, 4491, 6131, CVE-2017-7303
Fedora 25	CVE-2016-4487, 4489, 4490, 4491, 6131, CVE-2017-7303
Fedora 26	CVE-2017-7303

Many Linux distributions have multiple versions affected by the vulnerabilities in our dataset, and we successfully generated PoCs for these versions. For example, OpenWRT is only affected by CVE-2017-7303 since its oldest version, OpenWRT 16, uses Binutils 2.28. In contrast, other Linux distributions have numerous versions impacted by multiple vulnerabilities due to their longer histories.

TransferFuzz cannot verify these vulnerabilities due to missing source code. The results demonstrate that TransferFuzz-Pro can generate PoCs for real-world binaries, even in the absence of source code, by leveraging binary instrumentation. Furthermore, the widespread reuse of vulnerable code significantly extends the impact beyond the software listed in CVE reports.

Answering RQ5: TransferFuzz-Pro expanded the impacted software number listed in CVE reports from 15 to 53 and generated corresponding PoC for different real-world impacted Linux distributions.

VI. THREATS TO VALIDITY

One concern is the effectiveness of TransferFuzz-Pro. Not all vulnerabilities allow for the extraction of key-byte traces, as observed with CVE-2017-7303. However, many vulnerabilities do produce such traces, as demonstrated by the Binutils dataset, where all binaries contained key-byte traces, significantly improving experimental performance. Additionally, some binaries may slow down the fuzzing process due to complex branch constraints or missing key-byte traces. Despite these challenges, TransferFuzz-Pro consistently completed vulnerability verifications within two hours, with over half of the cases resolved in under one minute. This highlights its effectiveness in verifying vulnerability code propagation.

The second concern involves implementation obstacles. TransferFuzz-Pro is the first work to automate processes through option combination generation, making it accessible to users without expertise in software analysis or vulnerabilities. Furthermore, its binary-level instrumentation ensures fuzzing is independent of source code. However, the option combination generation module relies on source code for analysis. We used historical results from the source code to generate PoCs for binaries across different Linux distributions. For binaries lacking source code, pseudocode obtained through decompilation can serve as a substitute, but this approach significantly reduces accuracy. While TransferFuzz-Pro demonstrates a high degree of automation and usability, addressing this issue will be reserved for future research.

The third concern is the evaluation dataset. The dataset for evaluation was selected not to intentionally showcase TransferFuzz-Pro's strengths but to maintain objectivity. It encompasses all instances from AFLGo, SelectFuzz and DAFL related to vulnerability code propagation, alongside as many cases as possible from OCTOPOCs (their dataset not being available). Our objective was to employ unbiased datasets for existing methods. However, further empirical studies across a wider range of projects are essential to fully generalize our evaluation results.

VII. RELATED WORKS

A. Code Reuse Detection

A substantial body of work has been developed for code reuse detection. Due to differences in compilation options and architectures, the same source code can produce significantly different binaries, making binary similarity analysis a challenging task [34].

One category of approaches relies on constant features to detect code reuse. These methods identify third-party library (TPL) reuse by extracting identical constant features, such as

strings [35], [36], function names [8], and jump tables [9], from both the target binary and the source binary.

Another category employs binary code similarity detection techniques. Tools like Gitz [37] and VIVA [38] use text hashing to represent function features. Symbolic execution-based methods, such as BinSim [39] and Bingo [40], provide a more robust solution for detecting function similarity. Additionally, deep learning-based approaches [26], [41] learn function semantics to improve similarity detection.

Some methods focus on code granularity to enhance code reuse detection. ModX [42] clusters functions with similar functionality and sets thresholds for comparison within each group. LibDB [43] refines function-matching results into mini-function call graphs (mini-FCGs) to detect code reuse. LibAM [5] performs similarity checks between function regions to identify reused code within the target binary.

While these techniques are effective at detecting reused code, they do not validate whether the reused code contains triggerable vulnerabilities. TransferFuzz-Pro can complement these approaches by further analyzing the detected reused code, filtering out false positives, and verifying the presence of vulnerabilities.

B. Vulnerability Detection

Vulnerability detection has been a critical area of research in computer science and cybersecurity, focusing on identifying vulnerabilities in newly developed code by extracting features from known vulnerable functions and determining their presence in target code.

Early works, such as Bingo [40] and SAFE [44], calculated function similarity by directly comparing vulnerable functions to all functions in the target code. More recent approaches, including MVP [45] and VIVA [38], use data stream slicing techniques to detect vulnerabilities by identifying the presence of vulnerable code and the absence of corresponding patch code. Additionally, tools like Fiber [11] and PDiff [12] leverage symbolic execution to extract deep patch semantics, determining whether a target vulnerable function has been patched.

However, these methods focus solely on detecting the existence of vulnerabilities or patches and do not assess whether a vulnerability is actually triggerable in a new context. TransferFuzz-Pro addresses this limitation by not only detecting vulnerabilities but also verifying their triggerability, effectively bridging this gap.

C. Directed Grey-Box Fuzzing

Fuzzing is a critical technique for discovering vulnerabilities in real-world software, including parsers [46], network protocols [47], [48], web browsers [49], [50], mobile applications [51], and operating system kernels [52], [53], [54]. Significant advancements have been made in Grey-box fuzzing [55], [56], [57], which has become a dominant approach in the field.

Directed Grey-box Fuzzing (DGF) focuses on the challenge of generating test cases aimed at uncovering specific program bugs. AFLGo [15], one of the earliest directed fuzzers, prioritizes test case generation by favoring nodes closer to the target node in the Control-Flow Graph (CFG). Subsequent works, such as Hawkeye [58] and WindRanger [18], improve upon AFLGo by offering more accurate distance computations and enhanced feedback mechanisms.

Some fuzzers leverage historical fuzzing data to improve the efficiency of new fuzzing tasks. FuzzGuard [24] utilizes historical data from the same software to guide fuzzing efforts, while VulScope [25] extends this concept by using historical fuzzing data from different versions of the same software. In contrast, TransferFuzz-Pro introduces a novel approach by utilizing historical fuzzing data across different software, significantly enhancing the capabilities of directed fuzzing and improving its effectiveness.

D. Large Language Model for Software Security

In recent years, artificial intelligence has advanced rapidly, with large language models (LLMs) and agents attracting significant research attention. Multi-agent orchestration platforms, such as CrewAI [59], enable collaboration among multiple agents to accomplish complex tasks.

In the domain of software security, agents with capabilities approaching human-level performance have emerged. For example, companies like XBOW [60] and RunSybile [61] employ AI agents to automate penetration testing. XBOW has demonstrated the ability to find and exploit vulnerabilities in 75% of web security benchmarks and to discover new vulnerabilities in web applications. Dropzone AI [62] utilizes agents to automate alert classification and other first-line tasks in security operations centers. As security agents become more sophisticated and are integrated into multi-agent solutions, they have the potential to fundamentally transform network attack and defense dynamics.

Distinct from existing work, our approach is the first to achieve automated directed fuzzing. By leveraging an LLM to iteratively invoke the debugger and analyze code, we construct a code debugging agent that enables novel, automated option generation.

VIII. Conclusion

This paper introduces TransferFuzz-Pro, a novel framework designed to overcome the limitations of traditional vulnerability verification methods. Building on our previous work, TransferFuzz, it integrates LLM-driven code debugging technology to replicate a human-like debugging process and automatically generate PoC using historical vulnerabilities. Incorporating binary-level instrumentation further expands the scope of vulnerability verification. Experimental results demonstrate that TransferFuzz-Pro significantly improves verification speed and can automatically verify vulnerabilities that traditional methods cannot handle. TransferFuzz-Pro increased the number of affected software instances for 15 vulnerabilities in CVE and successfully generated PoCs for various impacted Linux distributions.

ACKNOWLEDGMENT

We appreciate all the anonymous reviewers for their invaluable comments and suggestions. Any opinions, findings and conclusions in this paper are those of the authors and do not necessarily reflect the views of the funding agencies. Yimi Ren is the first corresponding author.

REFERENCES

[1] "GitHub." 2008. [Online]. Available: https://github.com/
[2] "SourceForge." 2003. [Online]. Available: https://sourceforge.net/
[3] "VCPKG." 2016. [Online]. Available: https://vcpkg.io/
[4] Synopsys, "Synopsys 2024 open source security and risk analysis report." 2024. [Online]. Available: https://www.blackduck.com/resources/analyst-reports/open-source-security-risk-analysis.html
[5] S. Li et al., "LIBAM: An area matching framework for detecting third-party libraries in binaries," 2023, *arXiv:2305.04026*.
[6] "CVE," 1999. [Online]. Available: https://cve.mitre.org/
[7] "NVD," 2005. [Online]. Available: https://nvd.nist.gov/
[8] R. Duan, A. Bijlani, M. Xu, T. Kim, and W. Lee, "Identifying open-source license violation and 1-day security risk at large scale," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 2169–2185.
[9] Z. Yuan et al., "B2SFinder: Detecting open-source software reuse in cots software," in *Proc. 34th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 1038–1049.
[10] B. Zhao et al., "A large-scale empirical analysis of the vulnerabilities introduced by third-party components in IoT firmware," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2022, pp. 442–454.
[11] H. Zhang and Z. Qian, "Precise and accurate patch presence test for binaries," in *Proc. 27th USENIX Secur. Symp. (USENIX Secur.)*, 2018, pp. 887–902.
[12] Z. Jiang et al., "PDIFF: Semantic-based patch presence testing for downstream kernels," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2020, pp. 1149–1163.
[13] C. Dong et al., "LIBVDIFF: Library version difference guided OSS version identification in binaries," in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, New York, NY, USA: ACM, 2024, doi: 10.1145/3597503.3623336.
[14] S. Yang et al., "Towards practical binary code similarity detection: Vulnerability verification via patch semantic analysis," *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 6, pp. 1–29, Sep. 2023, doi: 10.1145/3604608.
[15] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 2329–2344.
[16] M.-D. Nguyen, S. Bardin, R. Bonichon, R. Groz, and M. Lemerre, "Binary-level directed fuzzing for Use-After-Free vulnerabilities," in *Proc. 23rd Int. Symp. Res. Attacks, Intrusions Defenses (RAID)*, 2020, pp. 47–62.
[17] H. Huang, Y. Guo, Q. Shi, P. Yao, R. Wu, and C. Zhang, "BEACON: Directed grey-box fuzzing with provable path pruning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 36–50.
[18] Z. Du, Y. Li, Y. Liu, and B. Mao, "WindRanger: A directed greybox fuzzer driven by deviation basic blocks," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 2440–2451.
[19] S. Li et al., "TransferFuzz: Fuzzing with historical trace for verifying propagated vulnerability code," 2024, *arXiv:2411.18347*.
[20] T. E. Kim, J. Choi, K. Heo, and S. K. Cha, "DAFL: Directed grey-box fuzzing guided by data dependency," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur.)*, 2023, pp. 4931–4948.
[21] C. Luo, W. Meng, and P. Li, "SelectFuzz: Efficient directed fuzzing with selective path exploration," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2023, pp. 2693–2707.
[22] D. Wang, Y. Li, Z. Zhang, and K. Chen, "CarpetFuzz: Automatic program option constraint extraction from documentation for fuzzing," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur.)*, 2023, pp. 1919–1936.
[23] D. Wang, G. Zhou, L. Chen, D. Li, and Y. Miao, "ProphetFuzz: Fully automated prediction and fuzzing of high-risk option combinations with only documentation via large language model," 2024, *arXiv:2409.00922*.
[24] P. Zong, T. Lv, D. Wang, Z. Deng, R. Liang, and K. Chen, "FuzzGuard: Filtering out unreachable inputs in directed grey-box fuzzing through deep learning," in *Proc. 29th USENIX Secur. Symp. (USENIX Secur.)*, 2020, pp. 2255–2269.
[25] J. Dai et al., "Facilitating vulnerability assessment through POC migration," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, New York, NY, USA: ACM, 2021, pp. 3300–3317, doi: 10.1145/3460120.3484594.
[26] H. Wang et al., "JTRANS: Jump-aware transformer for binary code similarity detection," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2022, pp. 1–13.
[27] S. Yang, L. Cheng, Y. Zeng, Z. Lang, H. Zhu, and Z. Shi, "ASTERIA: Deep learning-based AST-encoding for cross-platform binary code similarity detection," in *Proc. 51st Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 224–236.
[28] S. Kwon, S. Woo, G. Seong, and H. Lee, "Octopocs: Automatic verification of propagated vulnerable code using reformed proofs of concept," in *Proc. 51st Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 174–185.
[29] S. Kirkpatrick, C. D. Gelatt Jr., and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
[30] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in *Proc. 14th USENIX Workshop Offensive Technol. (WOOT)*, 2020, pp. 1–12.
[31] C. Aschermann, S. Schumilo, T. Blazytko, R. Gawlik, and T. Holz, "Redqueen: Fuzzing with input-to-state correspondence," in *Proc. NDSS*, vol. 19, pp. 1–15, 2019.
[32] "GDB." 1986. [Online]. Available: https://www.gnu.org/software/gdb/
[33] "PIN." 2017. [Online]. Available: https://software.intel.com/sites/landingpage/pintool/docs/98484/Pin/html/index.html
[34] I. U. Haq and J. Caballero, "A survey of binary code similarity," *ACM Comput. Surv.*, vol. 54, no. 3, pp. 1–38, 2021.
[35] A. Hemel, K. T. Kalleberg, R. Vermaas, and E. Dolstra, "Finding software license violations through binary code clone detection," in *Proc. 8th Work. Conf. Mining Softw. Repositories*, 2011, pp. 63–72.
[36] W. Tang, P. Luo, J. Fu, and D. Zhang, "LIBDX: A cross-platform and accurate system to detect third-party libraries in binary code," in *Proc. IEEE 27th Int. Conf. Softw. Anal., Evol. ReEng. (SANER)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 104–115.
[37] Y. David, N. Partush, and E. Yahav, "Similarity of binaries through re-optimization," in *Proc. 38th ACM SIGPLAN Conf. Program. Lang. Des. Implementation*, 2017, pp. 79–94.
[38] Y. Xiao et al., "Viva: Binary level vulnerability identification via partial signature," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. ReEng. (SANER)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 213–224.
[39] J. Ming, D. Xu, Y. Jiang, and D. Wu, "BinSim: Trace-based semantic binary diffing via system call sliced segment equivalence checking," in *Proc. 26th USENIX Secur. Symp. (USENIX Secur.),* 2017, pp. 253–270.
[40] M. Chandramohan, Y. Xue, Z. Xu, Y. Liu, C. Y. Cho, and H. B. K. Tan, "BinGo: Cross-architecture cross-OS binary search," in *Proc. 24th ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, 2016, pp. 678–689.
[41] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, "Neural network-based graph embedding for cross-platform binary code similarity detection," in *Proc. 2017 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 363–376.
[42] C. Yang, Z. Xu, H. Chen, Y. Liu, X. Gong, and B. Liu, "MODX: Binary level partially imported third-party library detection via program modularization and semantic matching," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 1393–1405.
[43] W. Tang, Y. Wang, H. Zhang, S. Han, P. Luo, and D. Zhang, "LIBDB: An effective and efficient framework for detecting third-party libraries in binaries," in *Proc. 19th Int. Conf. Mining Softw. Repositories*, 2022, pp. 423–434.
[44] L. Massarelli, G. A. D. Luna, F. Petroni, R. Baldoni, and L. Querzoni, "Safe: Self-attentive function embeddings for binary similarity," in *Proc. Int. Conf. Detection Intrusions Malware, Vulnerability Assessment*, New York, NY, USA: Springer, 2019, pp. 309–329.
[45] Y. Xiao et al., "MVP: Detecting vulnerabilities using Patch-Enhanced vulnerability signatures," in *29th USENIX Secur. Symp. (USENIX Secur.),* 2020, pp. 1165–1182.
[46] B. Mathis, R. Gopinath, M. Mera, A. Kampmann, M. Höschele, and A. Zeller, "Parser-directed fuzzing," in *Proc. 40th ACM Sigplan Conf. Program. Lang. Des. Implementation*, 2019, pp. 548–560.
[47] H. Gascon, C. Wressnegger, F. Yamaguchi, D. Arp, and K. Rieck, "PULSAR: Stateful black-box fuzzing of proprietary network protocols," in *Proc. Security Privacy Commun. Netw. 11th EAI Int.*

Conf., Dallas, TX, USA, 2015, New York, NY, USA: Springer, 2015, pp. 330–347.
[48] D. Liu, V.-T. Pham, G. Ernst, T. Murray, and B. I. Rubinstein, "State selection algorithms and their impact on the performance of stateful network protocol fuzzing," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. ReEng. (SANER)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 720–730.
[49] H. Han, D. Oh, and S. K. Cha, "CodealChemist: Semantics-aware code generation to find vulnerabilities in JavaScript engines," in *Proc. NDSS*, 2019, pp. 1–15.
[50] J. Wang, B. Chen, L. Wei, and Y. Liu, "Superion: Grammar-aware greybox fuzzing," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 724–735.
[51] L. Luo, Q. Zeng, B. Yang, F. Zuo, and J. Wang, "Westworld: Fuzzing-assisted remote dynamic symbolic execution of smart apps on IoT cloud platforms," in *Proc. Annu. Comput. Secur. Appl. Conf.*, 2021, pp. 982–995.
[52] J. Choi, K. Kim, D. Lee, and S. K. Cha, "NTFUZZ: Enabling type-aware kernel fuzzing on windows with static binary analysis," in *Proc. IEEE Symp. Secur. Privacy (SP)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 677–693.
[53] D. R. Jeong, K. Kim, B. Shivakumar, B. Lee, and I. Shin, "Razzer: Finding kernel race bugs through fuzzing," in *Proc. IEEE Symp. Secur. Privacy (SP)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 754–768.
[54] S. Schumilo, C. Aschermann, R. Gawlik, S. Schinzel, and T. Holz, "kAFL:"Hardware-Assisted," "feedback fuzzing for," OS "kernels," in *Proc. 26th USENIX Secur. Symp. (USENIX Secur.)*, 2017, pp. 167–182.
[55] M. Böhme, V. J. Manès, and S. K. Cha, "Boosting fuzzer efficiency: An information theoretic perspective," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2020, pp. 678–689.
[56] A. Fioraldi, D. C. D'Elia, and E. Coppa, "Weizz: Automatic grey-box fuzzing for structured binary formats," in *Proc. 29th ACM SIGSOFT Int. Symp. Softw. testing Anal.*, 2020, pp. 1–13.
[57] V. J. Manès et al., "The art, science, and engineering of fuzzing: A survey," *IEEE Trans. Softw. Eng.*, vol. 47, no. 11, pp. 2312–2331, Nov. 2021.
[58] H. Chen et al., "Hawkeye: Towards a desired directed grey-box Fuzzer," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 2095–2108.
[59] "Crewai." 2023. [Online]. Available: https://www.crewai.com/
[60] "Xbow." 2024. [Online]. Available: https://xbow.com/
[61] "Runsybil." 2023. [Online]. Available: https://www.runsybil.com/
[62] "Cropzone AI." 2020. [Online]. Available: https://www.dropzone.ai/

Siyuan Li received the B.E. degree from Shandong University in 2020. He is currently working toward the Ph.D. degree with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China. His research mainly focuses on software supply chain security, vulnerability analysis and code intelligence. He has published some top-tier conference/journal papers relevant to software engineering in ICSE, TOSEM, and NDSS.

Kaiyu Xie received the bachelor's degree in information security from Xinjiang University in 2022. He is currently working toward the Ph.D. degree in cyberspace security with the Institute of Information Engineering, Chinese Academy of Sciences. His research interests include software testing technology, LLM for software security, and vulnerability analysis.

Yuekang Li received the Ph.D. degree from Nanyang Technological University in 2020. Currently, he is an ARC DECRA Fellow and an Assistant Professor with the University of New South Wales. His research interest mainly lies in the field of software engineering and LLM-related research. He has published over 60 papers in journals and conferences including ICSE, ISSTA, ASE, ESEC/FSE, NDSS, S&P, Usenix Security, WWW, and TDSC.

Hong Li received the Ph.D. degree from the University of Chinese Academy of Sciences, Beijing, China, in 2017. Currently, he is a Professor with the Institute of Information Engineering, Chinese Academy of Sciences. His research mainly focuses on software supply chain security and Internet of Things security. He has published over 40 papers in journals and conferences including NDSS, Usenix Security, S&P, ICSE, TOSEM, AAAI, and INFOCOM.

Yimo Ren received the B.S. and M.S. degrees from Tianjin University, China, in 2018, and the Ph.D. degree from the Institute of Information Engineering, Chinese Academy of Sciences, China, in 2024. He is an Associate Researcher with the Institute of Information Engineering. His research interests include network security and artificial intelligence.

Limin Sun is a Professor with the Institute of Information Engineering, Chinese Academy of Sciences, China. He is also the Secretary General of the Select Committee of CWSN and the Director of Beijing Key Laboratory of IoT Information Security Technology. His main research interests include internet-of-things (IoT) security and industrial control system security. He is an Editor of the *Journal of Computer Science* and *Journal of Computer Applications*.

Hongsong Zhu (Member, IEEE) is a Professor with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China. His research mainly focuses on internet-of-things (IoT) security and AI embedded cyber security. He has published over 130 papers in journals and conferences including USENIX Security, IEEE TRANSACTIONS ON MOBILE COMPUTING, TOSEM, ICSE, EMNLP, ACL, and COLING.